

Laporan Akhir Tugas Besar
Pemrograman Berorientasi Objek
Klinikku - Aplikasi Manajemen Jadwal Pasien Pada
Klinik



Alif Wide Ardiansyah (1301223491)

S1 Informatika
Fakultas Informatika
Universitas Telkom
2026

DAFTAR ISI

HALAMAN JUDUL	1
DAFTAR ISI	2
DAFTAR TABEL	3
DAFTAR GAMBAR	5
1. PENDAHULUAN	7
1.1 LATAR BELAKANG	7
1.2 TUJUAN	7
1.3 DESKRIPSI UMUM SISTEM / RUANG LINGKUP	7
1.4 METODOLOGI PENGEMBANGAN SISTEM	8
2. DASAR TEORI	8
2.1 PEMROGRAMAN BERORIENTASI OBJEK	8
2.2 UNIFIED MODELING LANGUAGE (UML)	8
2.3 JAVA	8
2.4 POSTGRESQL	8
2.5 JAVA SPRING BOOT	8
2.6 JAVASCRIPT / TYPESCRIPT	9
2.7 REACT	9
2.8 TANSTACK START	9
3. ANALISIS DAN IMPLEMENTASI SISTEM	10
3.1 ANALISIS KEBUTUHAN SISTEM	10
3.2 IMPLEMENTASI BASIS DATA	10
3.3 IMPLEMENTASI UML	18
3.4 IMPLEMENTASI ANTARMUKA PENGGUNA	20
4. PENGUJIAN SISTEM	26

4.1 TUJUAN PENGUJIAN	26
4.2 LINGKUNGAN PENGUJIAN	26
4.3 METODE PENGUJIAN	27
5. HASIL UJI	29
5.1 FITUR LOGIN	29
5.2 FITUR LOGOUT	30
5.3 FITUR CRUD PASIEN	31
5.4 FITUR CRUD DOKTER	31
5.5 FITUR JADWAL DOKTER	32
5.6 FITUR APPOINTMENT	32
5.7 FITUR REKAM MEDIS	33
5.8 FITUR PEMBAYARAN	33
6. PENUTUP	34
DAFTAR PUSTAKA	35
LAMPIRAN	36
LAMPIRAN A PEMBAGIAN TUGAS	36
LAMPIRAN B SOURCE CODE PENTING	36
LAMPIRAN C SCREENSHOT SISTEM	37
LAMPIRAN D MANUAL INSTALASI SISTEM	42

DAFTAR TABEL

TABEL 1	TABEL ENTITAS USER	17
TABEL 2	TABEL ENTITAS ADMIN	17
TABEL 3	TABEL ENTITAS DOCTORS	18
TABEL 4	TABEL ENTITAS PATIENT	18
TABEL 5	TABEL ENTITAS DOCTOR SCHEDULES	18–19
TABEL 6	TABEL ENTITAS APPOINTMENT	19
TABEL 7	TABEL ENTITAS MEDICAL RECORD	19–20
TABEL 8	TABEL ENTITAS PAYMENT	20–21
TABEL 9	SKENARIO PENGUJIAN	30–32
TABEL 10	KASUS UJI FITUR LOGIN	32–33
TABEL 11	HASIL UJI FITUR LOGIN	33
TABEL 12	KASUS UJI FITUR LOGOUT	33
TABEL 13	HASIL UJI FITUR LOGOUT	33–34
TABEL 14	KASUS UJI FITUR CRUD PASIEN	34
TABEL 15	HASIL UJI FITUR CRUD PASIEN	34
TABEL 16	KASUS UJI FITUR CRUD DOKTER	34
TABEL 17	HASIL UJI FITUR CRUD DOKTER	35
TABEL 18	KASUS UJI FITUR JADWAL DOKTER	35
TABEL 19	HASIL UJI FITUR JADWAL DOKTER	35
TABEL 20	KASUS UJI FITUR APPOINTMENT	35–36

TABEL 21	HASIL UJI FITUR APPOINTMENT	36
TABEL 22	KASUS UJI FITUR REKAM MEDIS	36
TABEL 23	HASIL UJI FITUR REKAM MEDIS	36
TABEL 24	KASUS UJI FITUR PEMBAYARAN	37
TABEL 25	HASIL UJI FITUR PEMBAYARAN	37
TABEL 26	PEMBAGIAN TUGAS	39

DAFTAR GAMBAR

GAMBAR 1	ERD SISTEM	14–15
GAMBAR 2	CLASS DIAGRAM	21
GAMBAR 3	HALAMAN LOGIN	23
GAMBAR 4	HALAMAN DASHBOARD ADMIN	23
GAMBAR 5	DAFTAR DOKTER	24
GAMBAR 6	DAFTAR PASIEN	24
GAMBAR 7	APPOINTMENT ADMIN	25
GAMBAR 8	JADWAL DOKTER ADMIN	25
GAMBAR 9	REKAM MEDIS ADMIN	26
GAMBAR 10	PEMBAYARAN ADMIN	26
GAMBAR 11	HALAMAN DASHBOARD DOKTER	27
GAMBAR 12	APPOINTMENT DOKTER	27
GAMBAR 13	REKAM MEDIS DOKTER	28
GAMBAR 14	HALAMAN DASHBOARD PASIEN	28
GAMBAR 15	JANJI TEMU PASIEN	28
GAMBAR 16	REKAM MEDIS PASIEN	29
GAMBAR 17	PEMBAYARAN PASIEN	29
GAMBAR 18	SOURCE CODE 1	39
GAMBAR 19	SOURCE CODE 2	39
GAMBAR 20	SOURCE CODE 3	40
GAMBAR 21	SCREENSHOT SISTEM 1	40
GAMBAR 22	SCREENSHOT SISTEM 2	41
GAMBAR 23	SCREENSHOT SISTEM 3	42
GAMBAR 24	SCREENSHOT SISTEM 4	42

GAMBAR 25	SCREENSHOT SISTEM 5	43
GAMBAR 26	SCREENSHOT SISTEM 6	43
GAMBAR 27	SCREENSHOT SISTEM 7	44
GAMBAR 28	SCREENSHOT SISTEM 8	44
GAMBAR 29	SCREENSHOT SISTEM 9	45
GAMBAR 30	SCREENSHOT SISTEM 10	45
GAMBAR 31	DOKUMENTASI API	46

1 Pendahuluan

1.1 Latar Belakang

Manajemen operasional klinik kesehatan sering kali melibatkan proses yang kompleks dan rentan terhadap kesalahan, terutama dalam hal penjadwalan janji temu, pengelolaan antrean, pencatatan rekam medis, hingga proses pembayaran. Keterbatasan sistem manual atau sistem terfragmentasi dapat menyebabkan antrean panjang, informasi pasien yang tidak akurat, dan kesulitan dalam melacak riwayat kunjungan. Untuk mengatasi tantangan ini, kami membangun aplikasi Sistem Manajemen Klinik bernama Klinik-Ku. Aplikasi ini dirancang sebagai solusi terintegrasi untuk mengelola seluruh aspek penting operasional klinik, mencakup manajemen data pasien, dokter, jadwal praktik, reservasi janji temu, sistem antrean, rekam kunjungan pasien, dan transaksi pembayaran. Pemilihan topik ini sangat relevan dengan mata kuliah Pemrograman Berorientasi Objek (PBO) karena menyediakan berbagai kesempatan untuk menerapkan konsep PBO secara mendalam melalui berbagai entitas yang dapat dimodelkan menjadi kelas-kelas natural.

1.2 Tujuan

Meningkatkan efisiensi manajemen operasional klinik melalui sistem penjadwalan janji temu yang terotomatisasi.

Mempercepat akses pasien dalam melakukan reservasi konsultasi dan memantau status antrean secara real-time.

Menjamin akurasi rekam medis dan data kunjungan pasien dengan sistem pencatatan yang terintegrasi.

Menyederhanakan proses pembayaran melalui berbagai metode transaksi digital yang aman dan terlacak.

1.3 Deskripsi Umum Sistem (Ruang Lingkup)

Ruang lingkup sistem Klinik-Ku mencakup:

Manajemen Akun dan Data Pengguna: Mengelola data Admin, Dokter, dan Pasien, termasuk fungsi autentikasi, login, profil, serta tambah, ubah, dan hapus data user.

Manajemen Jadwal Dokter: Pengaturan slot waktu praktik oleh admin, pemantauan jadwal oleh dokter, dan pengecekan ketersediaan jadwal oleh pasien.

Booking Janji Temu: Proses pemesanan konsultasi pasien dengan dokter dengan manajemen status (Menunggu, Disetujui, Selesai, Dibatalkan).

Rekam Medis: Pencatatan riwayat pemeriksaan (keluhan, diagnosis, tindakan) oleh dokter, yang dapat diakses sesuai hak akses.

Manajemen Pembayaran Konsultasi: Pencatatan tagihan dan status pembayaran (Belum Bayar, Lunas, Dibatalkan) untuk transaksi internal klinik.

1.4 Metodologi Pengembangan Sistem

1.4.1 Analisis Kebutuhan: Mengidentifikasi kebutuhan fungsional dan non-fungsional sistem untuk memastikan fitur dan spesifikasi teknis sesuai dengan tujuan pengembangan.

1.4.2 Desain UI/UX: Merancang antarmuka pengguna yang intuitif dan pengalaman pengguna yang efisien untuk admin, dokter, dan pasien.

1.4.3 Desain Struktur Data: Merancang skema basis data relasional untuk menyimpan data pengguna, jadwal, rekam medis, dan transaksi pembayaran.

1.4.4 Desain API: Merancang endpoint API untuk komunikasi antara frontend dan backend, memastikan pertukaran data yang aman dan terstruktur.

1.4.5 Implementasi API: Membangun backend menggunakan Java Spring Boot untuk melayani permintaan data dan logika bisnis aplikasi.

1.4.6 Implementasi Frontend: Membangun antarmuka web menggunakan React dan TanStack Start untuk interaksi pengguna.

1.4.7 Integrasi API Dengan Frontend: Menghubungkan frontend dengan backend melalui API untuk memastikan data tampil secara dinamis dan responsif.

2 Dasar Teori

2.1 Pemrograman Berorientasi Objek

Pemrograman Berorientasi Objek (PBO) adalah paradigma pemrograman yang berpusat pada objek daripada fungsi. PBO memiliki empat prinsip utama: Enkapsulasi (menyembunyikan detail internal), Inheritansi (pewarisan sifat antarkelas), Polimorfisme (kemampuan objek memiliki banyak bentuk), dan Abstraksi (menyederhanakan sistem kompleks).

2.2 Unified Modeling Language (UML)

UML adalah standar industri untuk memvisualisasikan arsitektur sistem perangkat lunak. Dalam aplikasi ini, UML digunakan untuk mendokumentasikan desain sistem, termasuk diagram kelas untuk memodelkan struktur data dan relasi antar entitas klinik.

2.3 Java

Java adalah bahasa pemrograman tingkat tinggi yang berorientasi objek, bersifat "Write Once, Run Anywhere" (WORA), dan memiliki manajemen memori yang kuat melalui Java Virtual Machine (JVM). Java digunakan sebagai bahasa utama untuk membangun backend aplikasi Klinik-Ku.

2.4 PostgreSQL

PostgreSQL adalah sistem manajemen basis data relasional objek (ORDBMS) yang tangguh dan ACID-compliant. Sistem ini mendukung kueri kompleks, tipe data canggih, dan skalabilitas tinggi, menjadikannya pilihan utama untuk penyimpanan data klinis yang memerlukan integritas tinggi.

2.5 Java Spring Boot

Spring Boot adalah kerangka kerja berbasis Java yang menyederhanakan pengembangan aplikasi enterprise dengan konfigurasi otomatis (auto-configuration) and dependensi yang terkelola. Spring Boot memudahkan pembuatan layanan backend yang stabil dan modular.

2.6 Javascript / Typescript

JavaScript adalah bahasa pemrograman standar untuk pengembangan web sisi klien dan server. TypeScript adalah superset dari JavaScript yang menambahkan sistem tipe statis, sehingga meningkatkan keamanan kode, mempermudah refactoring, dan mengurangi kesalahan pada pengembangan skala besar.

2.7 React

React adalah pustaka JavaScript untuk membangun antarmuka pengguna (UI) berbasis komponen yang deklaratif dan efisien. React memungkinkan pengembang membuat aplikasi web yang responsif dengan memperbarui tampilan secara dinamis berdasarkan perubahan data.

2.8 TanStack Start

TanStack Start adalah kerangka kerja full-stack modern yang dibangun di atas dasar ekosistem TanStack. Framework ini menyediakan solusi untuk routing, data loading, dan server functions yang dioptimalkan untuk performa tinggi dalam aplikasi web modern.

3 Analisis dan Implementasi Sistem

3.1 Analisis Kebutuhan Sistem

3.1.1 Kebutuhan Fungsional (Fitur)

- 3.1.1.1. Fitur Login (sebagai admin / dokter / pasien)
- 3.1.1.2. Fitur Logout (sebagai admin / dokter / pasien)
- 3.1.1.3. Fitur CRUD Pasien (oleh admin)
- 3.1.1.4. Fitur CRUD Dokter (oleh admin)
- 3.1.1.5. Fitur CRUD Jadwal (oleh admin)
- 3.1.1.6. Fitur CRUD Appointment (oleh admin)
- 3.1.1.7. Fitur Manajemen Appointment (oleh dokter)
- 3.1.1.8. Fitur Pembuatan Appointment (oleh pasien)
- 3.1.1.9. Fitur Pembuatan Rekam Medis (oleh dokter)
- 3.1.1.10. Fitur Melihat Rekam Medis (oleh admin / dokter / pasien)
- 3.1.1.11. Fitur Pembuatan Pembayaran (oleh admin)
- 3.1.1.12. Fitur Manajemen Status Pembayaran (oleh admin)
- 3.1.1.13. Fitur Melihat Pembayaran (oleh pasien)

3.1.2 Kebutuhan Non-Fungsional

3.1.2.1. Spesifikasi Perangkat Lunak

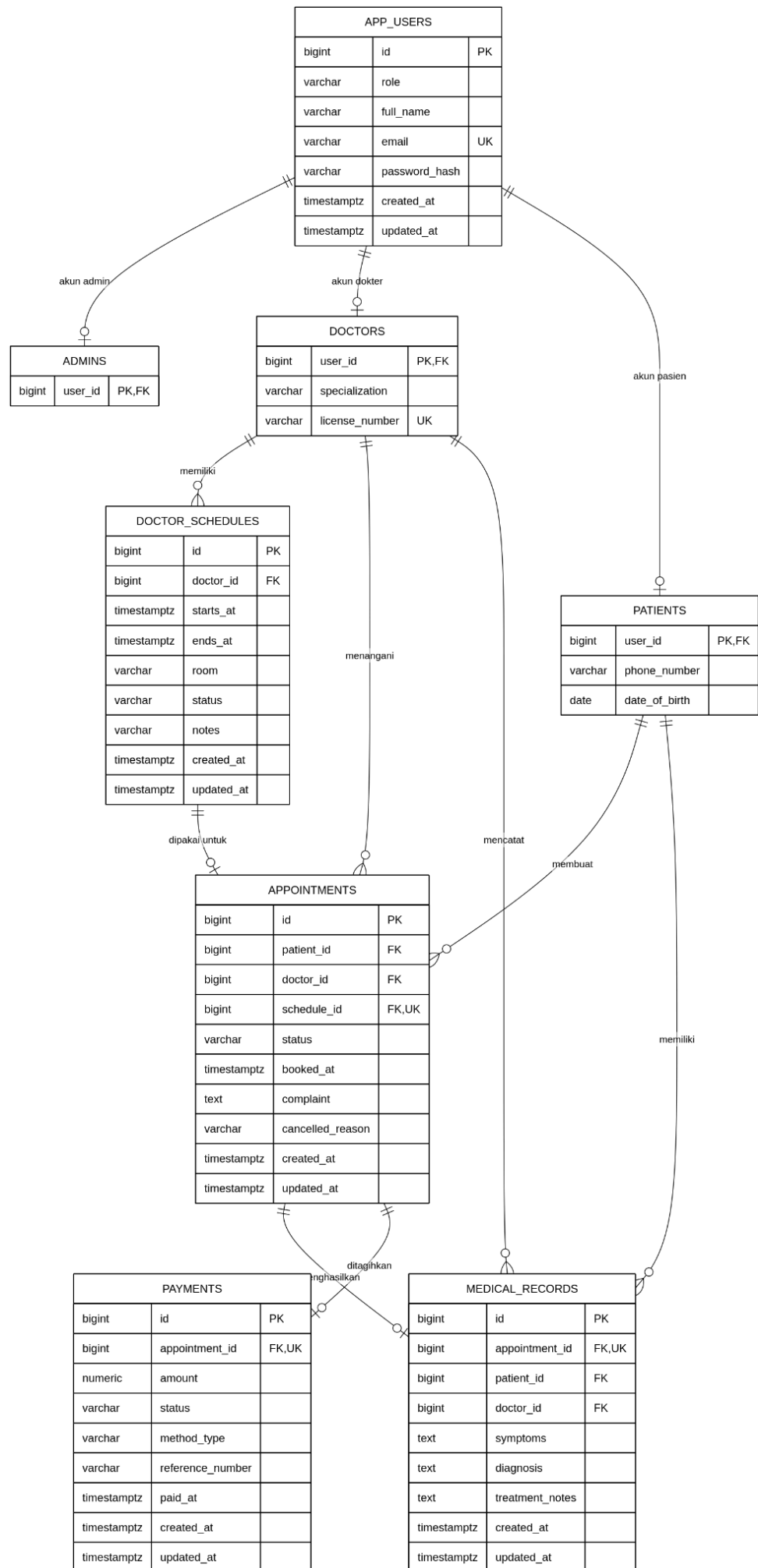
- a. Backend: Java Spring Boot
- b. Frontend: React / Tanstack Start
- c. Bahasa Pemrograman: Java dan JavaScript/TypeScript
- d. Database: PostgreSQL

3.1.2.2. Spesifikasi Perangkat Keras

- a. Processor: Minimal Intel Core i5 atau setara
- b. RAM: Minimal 8 GB
- c. Penyimpanan: Minimal 256 GB SSD

3.2 Implementasi Basis Data

3.2.1 Entity Relationship Diagram (ERD)



Entitas User

Menyimpan data pengguna pada sistem. Entitas ini memiliki atribut id sebagai primary key, role untuk menentukan peran pengguna, full_name untuk nama lengkap, email sebagai email login yang bersifat unik, password_hash untuk menyimpan password yang sudah di-hash, serta created_at dan updated_at untuk mencatat waktu pembuatan dan pembaruan data. Entitas ini berelasi dengan entitas admins, doctors, dan patients dengan kardinalitas 1 ke 0..1, karena satu akun pengguna hanya dapat memiliki satu data turunan sesuai perannya, yaitu admin, dokter, atau pasien.

Entitas Admin

Menyimpan data admin pada sistem. Entitas ini memiliki atribut user_id sebagai primary key sekaligus foreign key yang mengacu ke app_users.id. Entitas ini berelasi dengan entitas app_users dengan kardinalitas 0..1 ke 1, karena satu data admin pasti berasal dari satu akun pengguna, sedangkan satu akun pengguna belum tentu merupakan admin.

Entitas Doctors

Menyimpan data dokter pada sistem. Entitas ini memiliki atribut user_id sebagai primary key sekaligus foreign key yang mengacu ke app_users.id, specialization untuk menyimpan spesialisasi dokter, dan license_number untuk menyimpan nomor izin praktik dokter yang bersifat unik. Entitas ini berelasi dengan entitas app_users dengan kardinalitas 0..1 ke 1, karena satu dokter pasti memiliki satu akun pengguna. Entitas ini juga berelasi dengan entitas doctor_schedules dengan kardinalitas 1 ke banyak, karena satu dokter dapat memiliki banyak jadwal praktik. Selain itu, entitas ini berelasi dengan entitas appointments dengan kardinalitas 1 ke banyak, karena satu dokter dapat menangani banyak janji temu, serta berelasi dengan entitas medical_records dengan kardinalitas 1 ke banyak, karena satu dokter dapat mencatat banyak rekam medis.

Entitas Patients

Menyimpan data pasien pada sistem. Entitas ini memiliki atribut user_id sebagai primary key sekaligus foreign key yang mengacu ke app_users.id, phone_number untuk menyimpan nomor telepon pasien, dan date_of_birth untuk menyimpan tanggal lahir pasien. Entitas ini berelasi dengan entitas app_users dengan kardinalitas 0..1 ke 1, karena satu pasien pasti memiliki satu akun pengguna. Entitas ini juga berelasi dengan entitas appointments dengan kardinalitas 1 ke banyak, karena satu pasien dapat membuat banyak janji temu, serta berelasi dengan entitas medical_records dengan kardinalitas 1 ke banyak, karena satu pasien dapat memiliki banyak rekam medis.

Entitas Doctor Schedules

Menyimpan data jadwal praktik dokter. Entitas ini memiliki atribut id sebagai

primary key, doctor_id sebagai foreign key yang mengacu ke doctors.user_id, starts_at untuk waktu mulai jadwal, ends_at untuk waktu selesai jadwal, room untuk ruangan praktik, status untuk status jadwal, notes untuk catatan tambahan, serta created_at dan updated_at untuk mencatat waktu pembuatan dan pembaruan data. Entitas ini berelasi dengan entitas doctors dengan kardinalitas banyak ke 1, karena banyak jadwal dapat dimiliki oleh satu dokter. Entitas ini juga berelasi dengan entitas appointments dengan kardinalitas 1 ke 0..1, karena satu jadwal hanya dapat dipakai oleh maksimal satu janji temu.

Entitas Schedules

Menyimpan data janji temu antara pasien dan dokter. Entitas ini memiliki atribut id sebagai primary key, patient_id sebagai foreign key yang mengacu ke patients.user_id, doctor_id sebagai foreign key yang mengacu ke doctors.user_id, schedule_id sebagai foreign key yang mengacu ke doctor_schedules.id, status untuk status janji temu, booked_at untuk waktu pemesanan, complaint untuk keluhan pasien, cancelled_reason untuk alasan pembatalan, serta created_at dan updated_at untuk mencatat waktu pembuatan dan pembaruan data. Entitas ini berelasi dengan entitas patients dengan kardinalitas banyak ke 1, karena banyak janji temu dapat dibuat oleh satu pasien. Entitas ini berelasi dengan entitas doctors dengan kardinalitas banyak ke 1, karena banyak janji temu dapat ditangani oleh satu dokter. Entitas ini juga berelasi dengan entitas doctor_schedules dengan kardinalitas 0..1 ke 1, karena satu janji temu dapat menggunakan satu jadwal dokter atau tidak memiliki jadwal. Selain itu, entitas ini berelasi dengan entitas medical_records dengan kardinalitas 1 ke 0..1 dan dengan entitas payments dengan kardinalitas 1 ke 0..1, karena satu janji temu dapat menghasilkan satu rekam medis dan satu pembayaran.

Entitas Medical Records

Menyimpan data rekam medis pasien dari hasil janji temu. Entitas ini memiliki atribut id sebagai primary key, appointment_id sebagai foreign key unik yang mengacu ke appointments.id, patient_id sebagai foreign key yang mengacu ke patients.user_id, doctor_id sebagai foreign key yang mengacu ke doctors.user_id, symptoms untuk gejala pasien, diagnosis untuk hasil diagnosis, treatment_notes untuk catatan tindakan atau perawatan, serta created_at dan updated_at untuk mencatat waktu pembuatan dan pembaruan data. Entitas ini berelasi dengan entitas appointments dengan kardinalitas 0..1 ke 1, karena satu rekam medis hanya berasal dari satu janji temu, sedangkan satu janji temu belum tentu sudah memiliki rekam medis. Entitas ini juga berelasi dengan entitas patients dengan kardinalitas banyak ke 1, karena banyak rekam medis dapat dimiliki oleh satu pasien, serta berelasi dengan entitas doctors dengan kardinalitas banyak ke 1, karena banyak rekam medis dapat dicatat oleh satu dokter.

Entitas Payments

Menyimpan data pembayaran untuk janji temu. Entitas ini memiliki atribut id sebagai primary key, appointment_id sebagai foreign key unik yang mengacu ke appointments.id, amount untuk nominal pembayaran, status untuk status pembayaran, method_type untuk metode pembayaran, reference_number untuk nomor referensi pembayaran, paid_at untuk waktu pembayaran lunas, serta created_at dan updated_at untuk mencatat waktu pembuatan dan pembaruan data. Entitas ini berelasi dengan entitas appointments dengan kardinalitas 0..1 ke 1, karena satu pembayaran hanya terkait dengan satu janji temu, sedangkan satu janji temu belum tentu sudah memiliki pembayaran.

3.2.2 Struktur Basis data

3.2.2.1. Tabel User

Tabel 1 Tabel Entitas User

Field	Tipe	Key
ID	BIGINT	Primary Key
ROLE	VARCHAR(32)	-
FULL_NAME	VARCHAR(120)	-
EMAIL	VARCHAR(160)	Unique Key
PASSWORD_HASH	VARCHAR(255)	-
CREATED_AT	TIMESTAMP WITH TIME ZONE	-
UPDATED_AT	TIMESTAMP WITH TIME ZONE	-

3.2.2.2. Tabel Admin

Tabel 2 Tabel Entitas Admin

Field	Tipe	Key
USER_ID	BIGINT	Primary Key, Foreign Key

3.2.2.3. Tabel Doctors

Tabel 3 Tabel Entitas Doctors

Field	Type	Key
USER_ID	BIGINT	Primary Key, Foreign Key
SPECIALIZATION	VARCHAR(120)	-
LICENSE_NUMBER	VARCHAR(64)	Unique Key

3.2.2.4. Tabel Patient

Tabel 4 Tabel Entitas Patient

Field	Type	Key
USER_ID	BIGINT	Primary Key, Foreign Key
PHONE_NUMBER	VARCHAR(32)	-
DATE_OF_BIRTH	DATE	-

3.2.2.5. Tabel Doctor Schedules

Tabel 5 Tabel Entitas Doctor Schedules

Field	Type	Key
ID	BIGINT	Primary Key
DOCTOR_ID	BIGINT	Foreign Key
STARTS_AT	TIMESTAMP WITH TIME ZONE	-
ENDS_AT	TIMESTAMP WITH TIME ZONE	-
ROOM	VARCHAR(40)	-
STATUS	VARCHAR(24)	-

Field	Type	Key
NOTES	VARCHAR(255)	-
CREATED_AT	TIMESTAMP WITH TIME ZONE	-
UPDATED_AT	TIMESTAMP WITH TIME ZONE	-

3.2.2.6. Tabel Appointment

Tabel 6 Tabel Entitas Appointment

Field	Type	Key
ID	BIGINT	Primary Key
PATIENT_ID	BIGINT	Foreign Key
DOCTOR_ID	BIGINT	Foreign Key
SCHEDULE_ID	BIGINT	Foreign Key, Unique Key
STATUS	VARCHAR(24)	-
BOOKED_AT	TIMESTAMP WITH TIME ZONE	-
COMPLAINT	TEXT	-
CANCELLED_REASON	VARCHAR(255)	-
CREATED_AT	TIMESTAMP WITH TIME ZONE	-
UPDATED_AT	TIMESTAMP WITH TIME ZONE	-

3.2.2.7. Tabel Medical Record

Tabel 7 Tabel Entitas Medical Record

Field	Tipe	Key
ID	BIGINT	Primary Key
APPOINTMENT_ID	BIGINT	Foreign Key, Unique Key
PATIENT_ID	BIGINT	Foreign Key
DOCTOR_ID	BIGINT	Foreign Key
SYMPTOMS	TEXT	-
DIAGNOSIS	TEXT	-
TREATMENT_NOTES	TEXT	-
CREATED_AT	TIMESTAMP WITH TIME ZONE	-
UPDATED_AT	TIMESTAMP WITH TIME ZONE	-

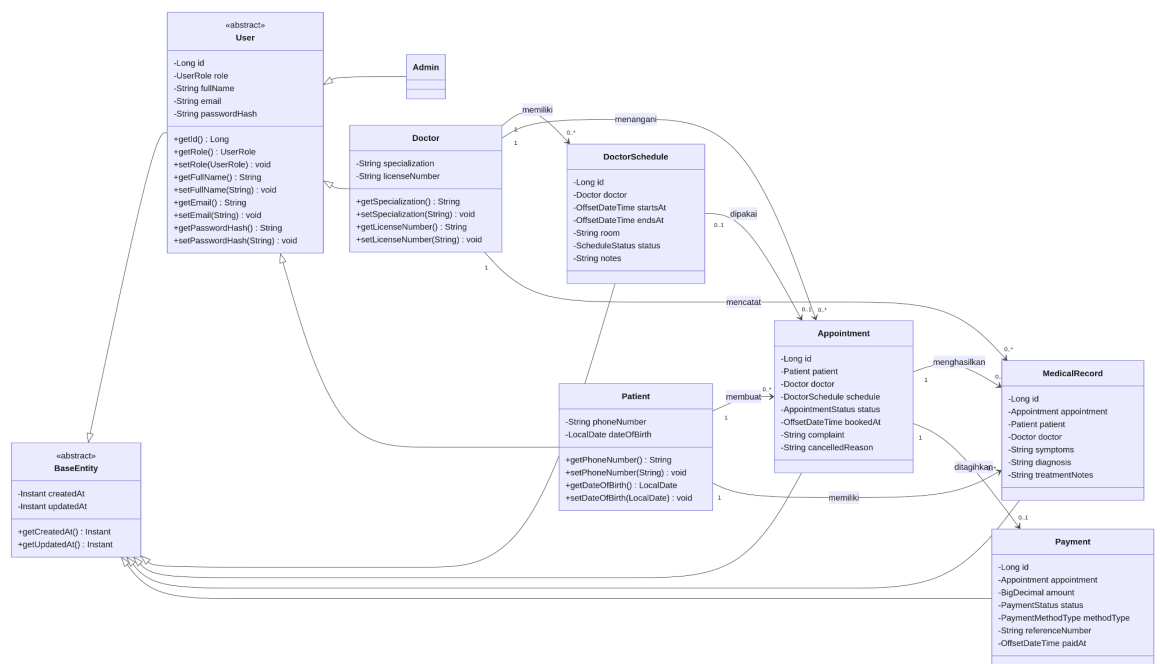
3.2.2.8. Tabel Payment

Tabel 8 Tabel Entitas Payment

Field	Tipe	Key
ID	BIGINT	Primary Key
APPOINTMENT_ID	BIGINT	Foreign Key, Unique Key
AMOUNT	NUMERIC(12, 2)	-
STATUS	VARCHAR(24)	-
METHOD_TYPE	VARCHAR(24)	-
REFERENCE_NUMBER	VARCHAR(100)	-
PAID_AT	TIMESTAMP WITH TIME ZONE	-

Field	Type	Key
CREATED_AT	TIMESTAMP WITH TIME ZONE	-
UPDATED_AT	TIMESTAMP WITH TIME ZONE	-

3.3 Implementasi UML Class Diagram



Gambar 2: Class Diagram

3.3.1 Deskripsi Kelas

3.3.1.1. Kelas Base Entity

Kelas BaseEntity merupakan kelas abstrak yang berfungsi sebagai superclass untuk menyimpan informasi waktu pembuatan dan perubahan data. Atribut utama pada class ini adalah createdAt dan updatedAt. Class ini diwarisi oleh class User, DoctorSchedule, Appointment, MedicalRecord, dan Payment.

3.3.1.2. Kelas User

Kelas User merupakan kelas abstrak yang berfungsi sebagai superclass untuk seluruh pengguna sistem. Class ini memiliki atribut id, role, fullName, email, dan passwordHash. Class User tidak digunakan sebagai objek langsung, tetapi diturunkan menjadi Admin, Doctor, dan Patient.

3.3.1.3. Kelas Admin

Kelas Admin merupakan turunan dari User yang merepresentasikan pengguna dengan peran administrator. Class ini mewarisi atribut dan method dari class User.

3.3.1.4. Kelas Doctor

Kelas Doctor merupakan turunan dari User yang merepresentasikan dokter. Class ini memiliki atribut tambahan specialization dan licenseNumber.

3.3.1.5. Kelas Patient

Kelas Patient merupakan turunan dari User yang merepresentasikan pasien. Class ini memiliki atribut tambahan phoneNumber dan dateOfBirth.

3.3.1.6. Kelas DoctorSchedule

Kelas DoctorSchedule digunakan untuk merepresentasikan jadwal praktik dokter. Atribut utama pada class ini adalah id, doctor, startsAt, endsAt, room, status, dan notes.

3.3.1.7. Kelas Appointment

Kelas Appointment digunakan untuk merepresentasikan janji temu antara pasien dan dokter. Atribut utama pada class ini adalah id, patient, doctor, schedule, status, bookedAt, complaint, dan cancelledReason.

3.3.1.8. Kelas Medicalrecord

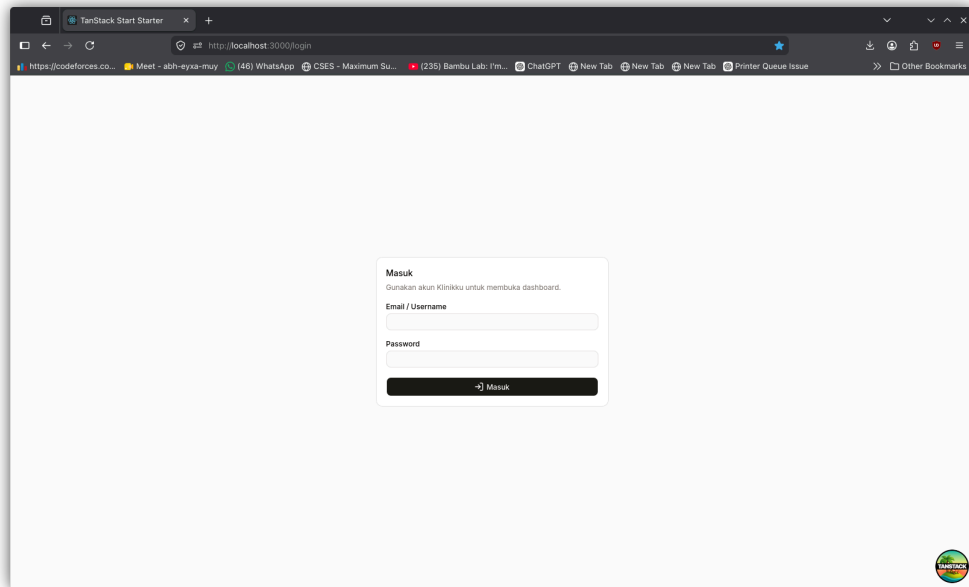
Kelas MedicalRecord digunakan untuk menyimpan data rekam medis dari appointment. Atribut utama pada class ini adalah id, appointment, patient, doctor, symptoms, diagnosis, dan treatmentNotes.

3.3.1.9. Kelas Payment

Kelas Payment digunakan untuk merepresentasikan pembayaran dari appointment. Atribut utama pada class ini adalah id, appointment, amount, status, methodType, referenceNumber, dan paidAt.

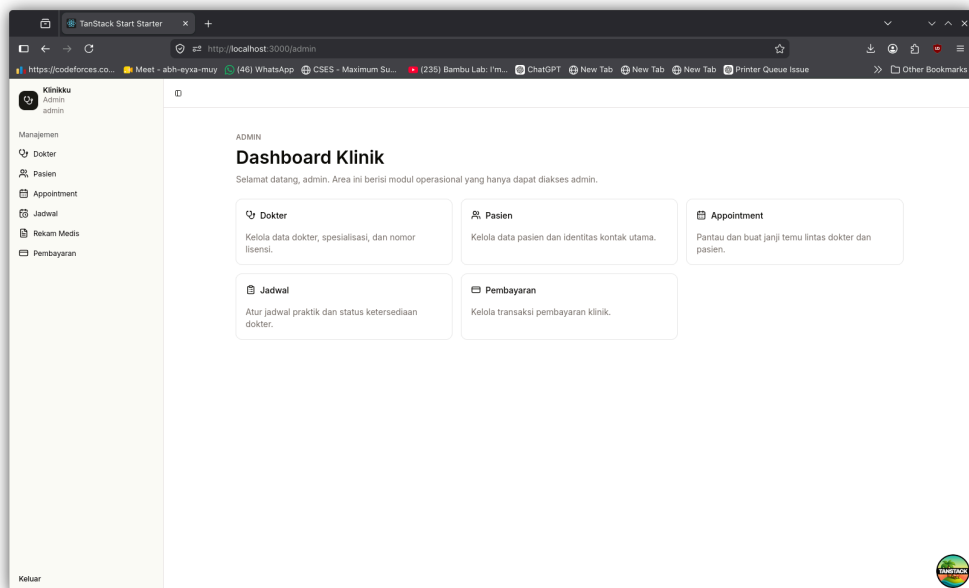
3.4 Implementasi Antarmuka Pengguna (*User Interface*)

3.4.1 Halaman *Login*



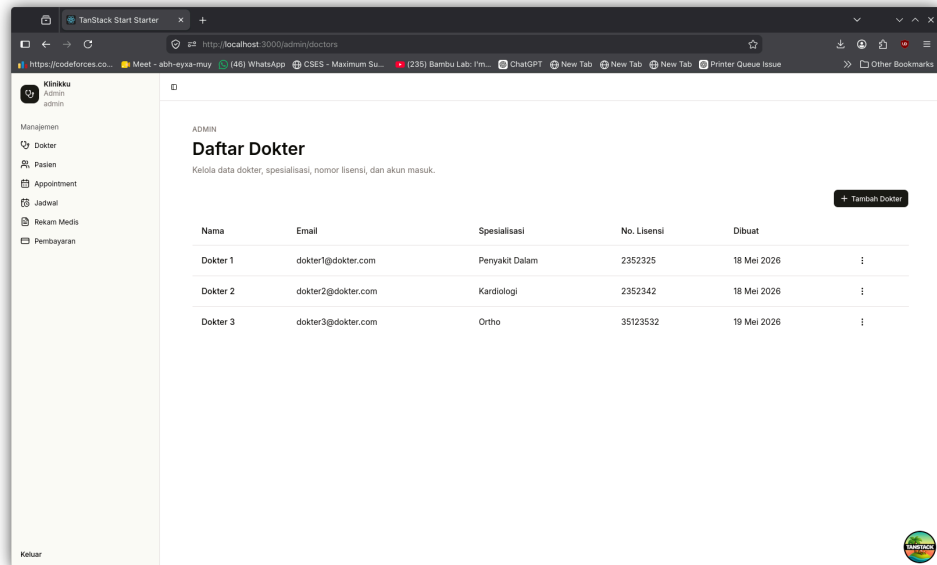
Gambar 3: Halaman Login

3.4.2 Halaman Dashboard Admin



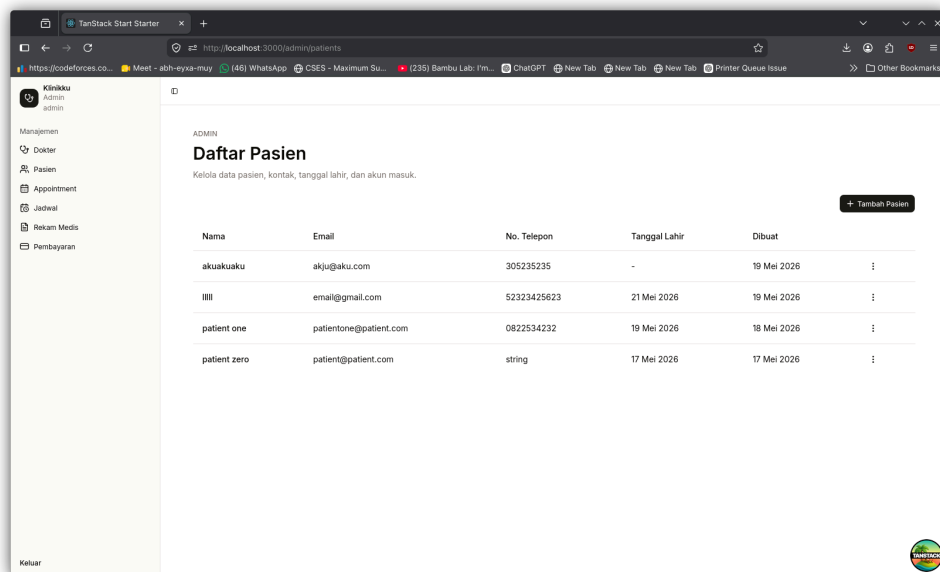
Gambar 4: Halaman Dashboard Admin

3.4.2.1. Daftar Dokter



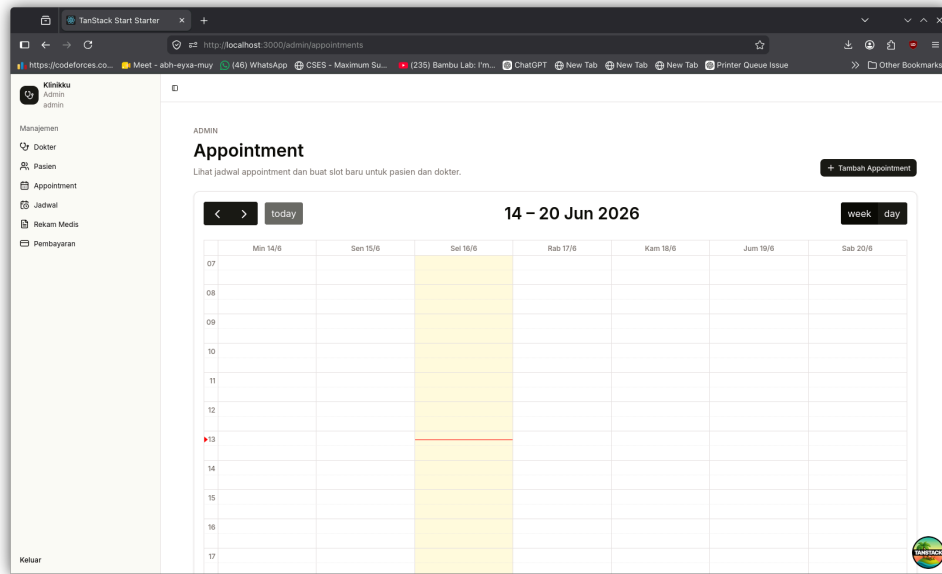
Gambar 5: Daftar Dokter

3.4.2.2. Daftar Pasien



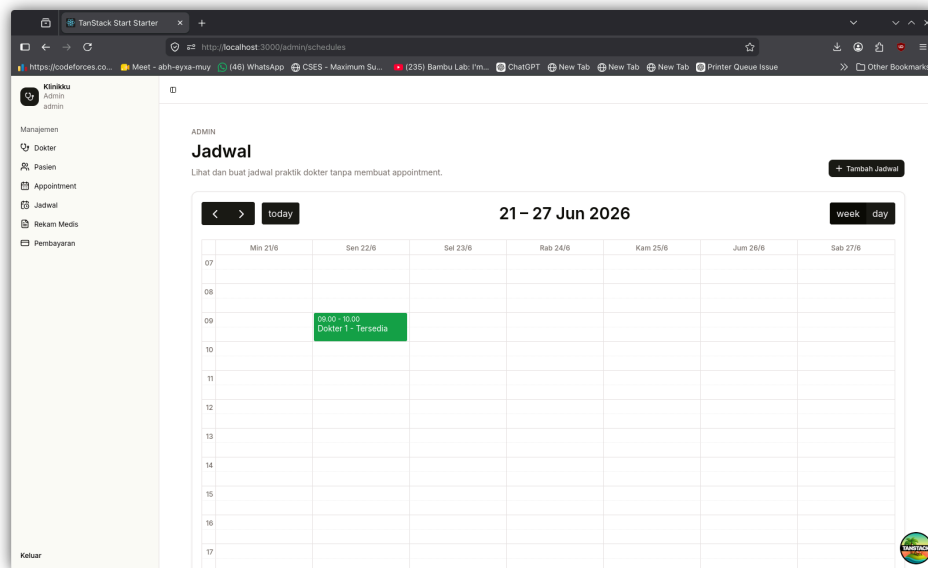
Gambar 6: Daftar Pasien

3.4.2.3. Appointment Dokter



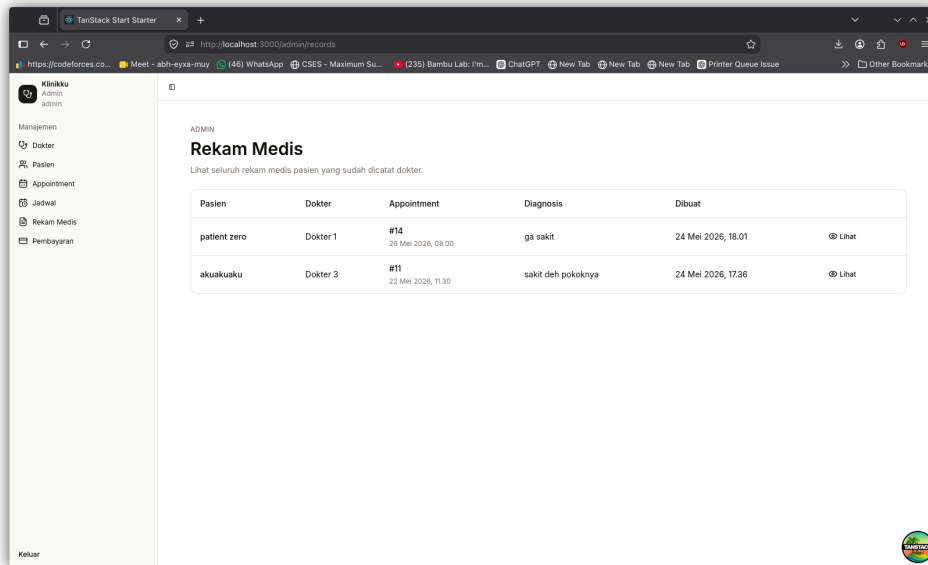
Gambar 7: Appointment Admin

3.4.2.4. Jadwal Dokter



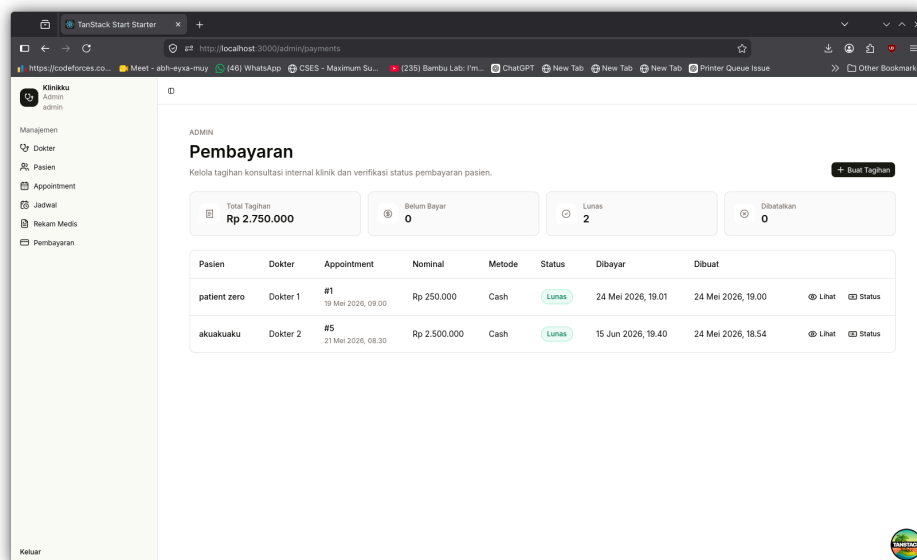
Gambar 8: Jadwal Dokter Admin

3.4.2.5. Rekam Medis



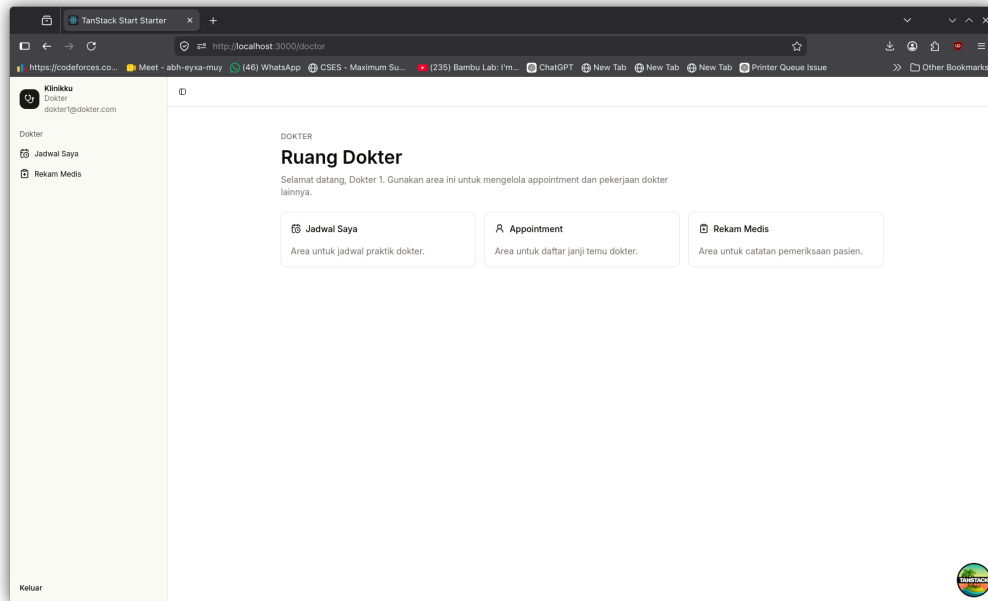
Gambar 9: Rekam Medis Admin

3.4.2.6. Pembayaran



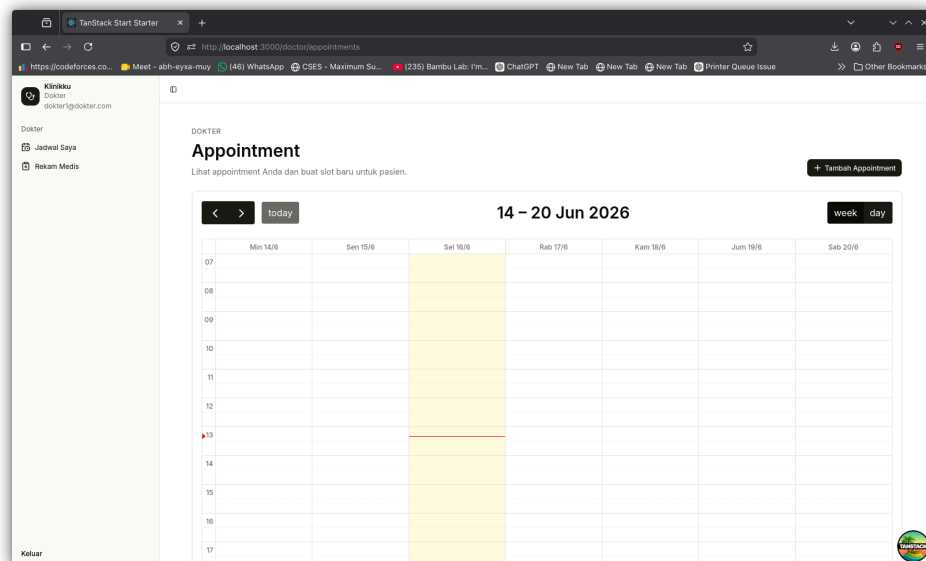
Gambar 10: Pembayaran Admin

3.4.3 Halaman Dashboard Dokter



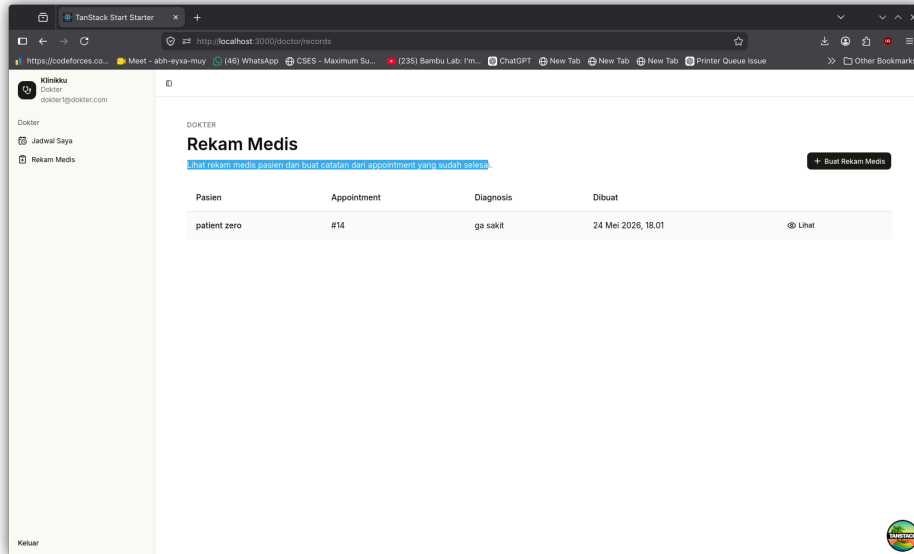
Gambar 11: Halaman Dashboard Dokter

3.4.3.1. Appointment



Gambar 12: Appointment Dokter

3.4.3.2. Rekam Medis

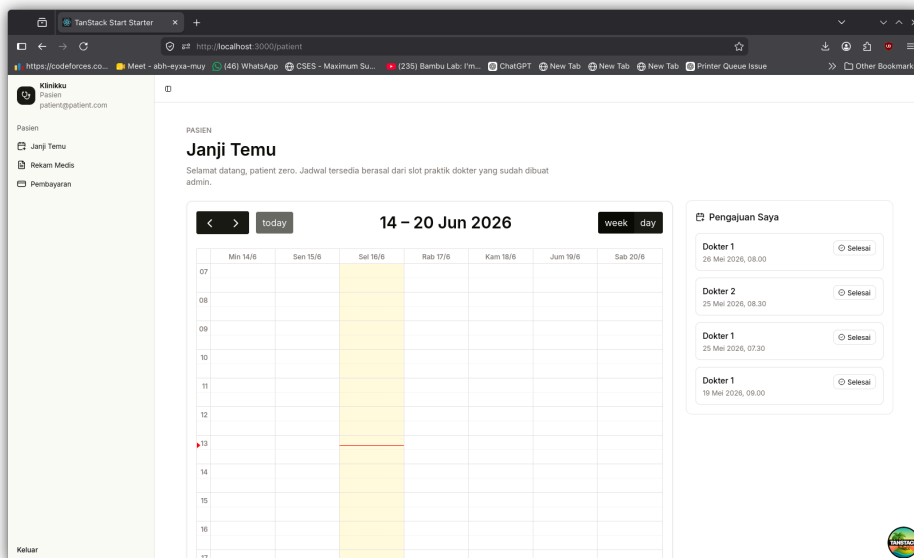


Gambar 13: Rekam Medis Dokter

3.4.4 Halaman Dashboard Pasien

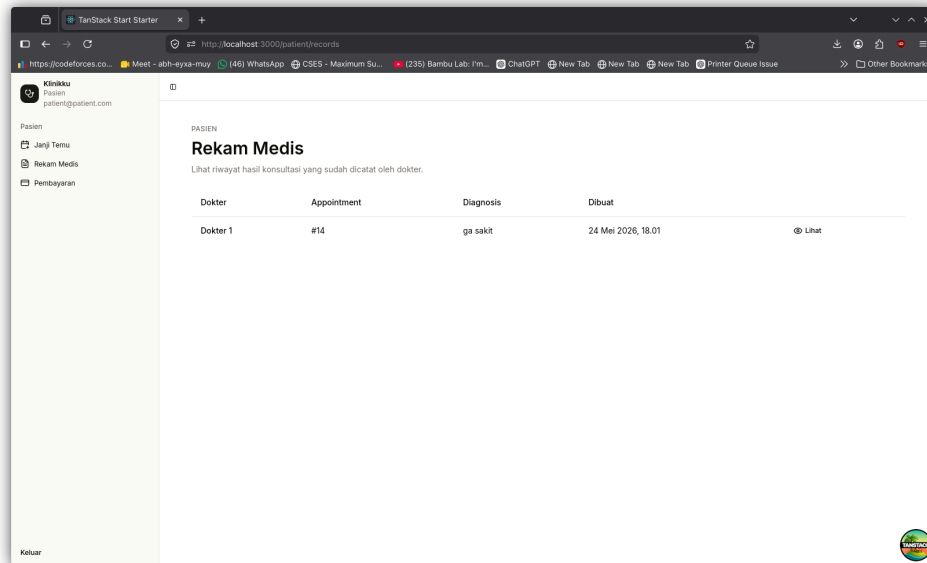
Gambar 14: Halaman Dashboard Pasien

3.4.4.1. Janji Temu



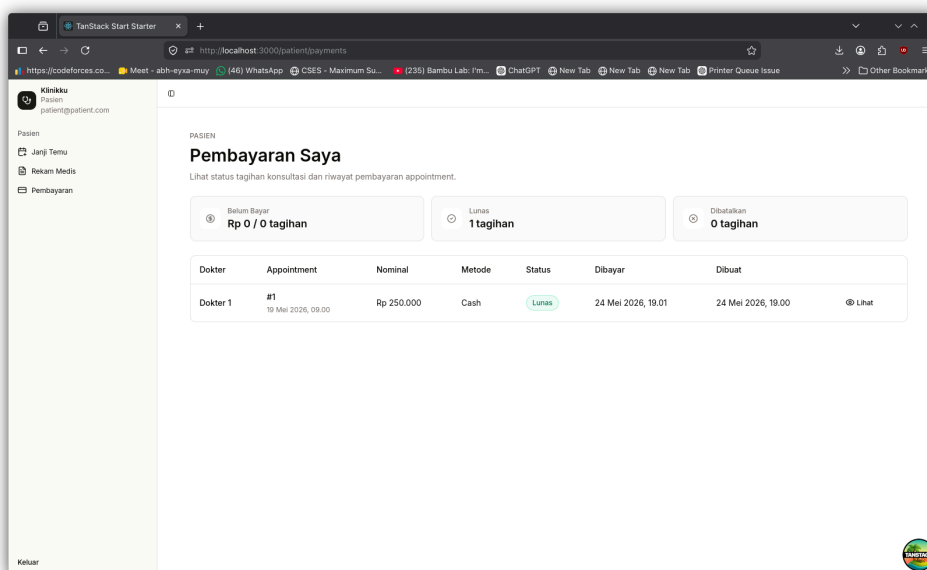
Gambar 15: Janji Temu Pasien

3.4.4.2. Rekam Medis



Gambar 16: Rekam Medis Pasien

3.4.4.3. Pembayaran



Gambar 17: Pembayaran Pasien

4 Pengujian Sistem

4.1 Tujuan Pengujian

4.2 Lingkungan Pengujian

4.2.1 Perangkat Lunak Pengujian

Sistem ini diujikan dengan spesifikasi perangkat lunak sebagai berikut;

- Sistem Operasi : Arch Linux - Firefox
- Bahasa Pemrograman : Typescript
- Database : Postgresql

4.2.2 Perangkat Keras Pengujian

Sistem ini diujikan dengan spesifikasi perangkat Keras sebagai berikut;

- Prosesor : *Ryzen 5 5600H*
- Memori : 32GB DDR4
- Storage : 512GB SSD

4.3 Metode Pengujian

4.3.1 Black Box Testing

Black Box Testing adalah metode pengujian perangkat lunak yang berfokus pada fungsionalitas sistem tanpa harus mengetahui struktur internal kode atau logika pemrograman di baliknya. Pengujian ini dilakukan dengan memberikan input tertentu pada sistem dan mengamati apakah output yang dihasilkan sesuai dengan spesifikasi kebutuhan yang telah ditetapkan.

Pada aplikasi Klinikku, Black Box Testing dilakukan dengan menguji seluruh fitur utama sistem melalui antarmuka pengguna (UI). Pengujian ini memastikan bahwa setiap modul, seperti login, CRUD data (pasien, dokter, jadwal), pembuatan appointment, rekam medis, dan pembayaran, berfungsi dengan benar sesuai dengan alur bisnis yang diharapkan. Fokus pengujian adalah pada kesesuaian hasil akhir yang terlihat oleh pengguna dengan kebutuhan fungsional sistem.

4.3.2 Skenario Pengujian

Pengujian dilakukan pada seluruh fitur utama sistem. Skenario pengujian dapat dilihat sebagai berikut

Tabel 9: Skenario Pengujian

Kelas Uji	Butir Uji	Identifikasi	Jenis Pengujian	Penguji
<i>Login</i>	Pengujian Fungsi <i>Login</i>	AO-LI01	Black Box	Alif
<i>Login</i>	Pengujian Kebenaran Fungsi <i>Login</i>	AO-LI02	Black Box	Alif
<i>Login</i>	Pengujian Validasi Field Kosong	AO-LI03	Black Box	Alif
<i>Logout</i>	Pengujian Fungsi <i>Logout</i>	AO-LO01	Black Box	Alif
<i>Logout</i>	Pengujian Akses Setelah <i>Logout</i>	AO-LO02	Black Box	Alif

Kelas Uji	Butir Uji	Identifikasi	Jenis Pengujian	Penguji
CRUD Pasien	Pengujian Fungsi Menambahkan Data Pasien	AO-PS01	Black Box	Alif
CRUD Pasien	Pengujian Fungsi Mengubah Data Pasien	AO-PS02	Black Box	Alif
CRUD Pasien	Pengujian Fungsi Menghapus Data Pasien	AO-PS03	Black Box	Alif
CRUD Dokter	Pengujian Fungsi Menambahkan Data Dokter	AO-DK01	Black Box	Alif
CRUD Dokter	Pengujian Fungsi Mengubah Data Dokter	AO-DK02	Black Box	Alif
CRUD Dokter	Pengujian Fungsi Menghapus Data Dokter	AO-DK03	Black Box	Alif
Jadwal Dokter	Pengujian Fungsi Menambahkan Jadwal Dokter	AO-JD01	Black Box	Alif
Jadwal Dokter	Pengujian Fungsi Mengubah Jadwal Dokter	AO-JD02	Black Box	Alif
Jadwal Dokter	Pengujian Jadwal Bentrok	AO-JD03	Black Box	Alif
Appointment	Pengujian Fungsi Pembuatan <i>Appointment</i> oleh Pasien	AO-AP01	Black Box	Alif
Appointment	Pengujian Perubahan Status	AO-AP02	Black Box	Alif

Kelas Uji	Butir Uji	Identifikasi	Jenis Pengujian	Penguji
	<i>Appointment</i> oleh Admin			
Appointment	Pengujian Perubahan Status <i>Appointment</i> oleh Dokter	AO-AP03	Black Box	Alif
Rekam Medis	Pengujian Fungsi Pembuatan Rekam Medis oleh Dokter	AO-RM01	Black Box	Alif
Rekam Medis	Pengujian Akses Rekam Medis oleh Admin / Dokter / Pasien	AO-RM02	Black Box	Alif
Pembayaran	Pengujian Fungsi Pembuatan Pembayaran oleh Admin	AO-PB01	Black Box	Alif
Pembayaran	Pengujian Perubahan Status Pembayaran oleh Admin	AO-PB02	Black Box	Alif
Pembayaran	Pengujian Akses Riwayat Pembayaran oleh Pasien	AO-PB03	Black Box	Alif

5 Hasil Uji

Hasil uji berdasarkan skenario yang telah dibuat pada bab sebelumnya.

5.1 Fitur *Login*

5.1.1 Kasus uji

Tabel 10: Kasus Uji Fitur *Login*

Identifikasi	Skenario
AO-LI01	Pengguna melakukan proses login ke dalam sistem.

AO-LI02	Pengguna melakukan login menggunakan data akun yang benar.
AO-LI03	Pengguna melakukan login dengan field yang masih kosong.

5.1.2 Hasil

Tabel 11: Hasil Uji Fitur Login

Identifikasi	Input	Hasil yang diharapkan	Hasil sebenarnya	Status
AO-LI01	Username dan password diisi	Sistem memproses login dan mengarahkan pengguna ke halaman utama sesuai role	Sistem berhasil memproses login dan mengarahkan pengguna ke halaman utama sesuai role	Sukses
AO-LI02	Username dan password benar	Sistem menerima data login dan pengguna berhasil masuk ke sistem	Sistem menerima data login dan pengguna berhasil masuk ke sistem	Sukses
AO-LI03	Username atau password kosong	Sistem menampilkan pesan validasi bahwa field wajib diisi	Sistem menampilkan pesan validasi bahwa field wajib diisi	Sukses

5.2 Fitur Logout

5.2.1 Kasus uji

Beberapa kasus uji yang dilakukan sebagai berikut;

Tabel 12: Kasus Uji Fitur Logout

Identifikasi	Skenario
AO-LO01	Pengguna melakukan proses logout dari sistem.
AO-LO02	Pengguna mencoba mengakses halaman sistem setelah logout.

5.2.2 Hasil

Hasil Uji Sebagai Berikut

Tabel 13: Hasil Uji Fitur Logout

Identifikasi	Input	Hasil yang diharapkan	Hasil sebenarnya	Status
AO-LO01	Pengguna menekan tombol logout	Sistem mengakhiri sesi pengguna dan mengarahkan ke halaman login	Sistem mengakhiri sesi pengguna dan mengarahkan ke halaman login	Sukses

AO-LO02	Pengguna mengakses halaman sistem setelah logout	Sistem menolak akses dan mengarahkan pengguna ke halaman login	Sistem menolak akses dan mengarahkan pengguna ke halaman login	Sukses
---------	--	--	--	--------

5.3 Fitur CRUD Pasien

5.3.1 Kasus Uji

Beberapa kasus uji yang dilakukan sebagai berikut;

Tabel 14: Kasus Uji Fitur CRUD Pasien

Identifikasi	Skenario
AO-PS01	Admin menambahkan data pasien baru.
AO-PS02	Admin mengubah data pasien yang sudah ada.
AO-PS03	Admin menghapus data pasien.

5.3.2 Hasil

Hasil Uji Sebagai Berikut

Tabel 15: Hasil Uji Fitur CRUD Pasien

Identifikasi	Input	Hasil yang diharapkan	Hasil sebenarnya	Status
AO-PS01	Data pasien diisi lengkap dan valid	Sistem menyimpan data pasien baru ke dalam database	Sistem berhasil menyimpan data pasien baru ke dalam database	Sukses
AO-PS02	Data pasien diperbarui	Sistem menyimpan perubahan data pasien	Sistem berhasil menyimpan perubahan data pasien	Sukses
AO-PS03	Data pasien dipilih untuk dihapus	Sistem menghapus data pasien dari database	Sistem berhasil menghapus data pasien dari database	Sukses

5.4 Fitur CRUD Dokter

5.4.1 Kasus Uji

Tabel 16: Kasus Uji Fitur CRUD Dokter

Identifikasi	Skenario
AO-DK01	Admin menambahkan data dokter baru.
AO-DK02	Admin mengubah data dokter yang sudah ada.
AO-DK03	Admin menghapus data dokter.

5.4.2 Hasil

Tabel 17: Hasil Uji Fitur CRUD Dokter

Identifikasi	Input	Hasil yang diharapkan	Hasil sebenarnya	Status
AO-DK01	Data dokter diisi lengkap dan valid	Sistem menyimpan data dokter baru ke dalam database	Sistem berhasil menyimpan data dokter baru ke dalam database	Sukses
AO-DK02	Data dokter diperbarui	Sistem menyimpan perubahan data dokter	Sistem berhasil menyimpan perubahan data dokter	Sukses
AO-DK03	Data dokter dipilih untuk dihapus	Sistem menghapus data dokter dari database	Sistem berhasil menghapus data dokter dari database	Sukses

5.5 Fitur Jadwal Dokter

5.5.1 Kasus Uji

Tabel 18: Kasus Uji Fitur Jadwal Dokter

Identifikasi	Skenario
AO-JD01	Admin menambahkan jadwal dokter baru.
AO-JD02	Admin mengubah jadwal dokter yang sudah ada.
AO-JD03	Admin menambahkan jadwal dokter yang bentrok.

5.5.2 Hasil

Tabel 19: Hasil Uji Fitur Jadwal Dokter

Identifikasi	Input	Hasil yang diharapkan	Hasil sebenarnya	Status
AO-JD01	Data jadwal dokter diisi lengkap dan valid	Sistem menyimpan jadwal dokter baru ke dalam database	Sistem berhasil menyimpan jadwal dokter baru ke dalam database	Sukses
AO-JD02	Data jadwal dokter diperbarui	Sistem menyimpan perubahan jadwal dokter	Sistem berhasil menyimpan perubahan jadwal dokter	Sukses
AO-JD03	Jadwal dokter diisi dengan waktu yang bentrok	Sistem menolak penyimpanan dan menampilkan pesan jadwal bentrok	Sistem menolak penyimpanan dan menampilkan pesan jadwal bentrok	Sukses

5.6 Fitur Appointment

5.6.1 Kasus Uji

Tabel 20: Kasus Uji Fitur Appointment

Identifikasi	Skenario
AO-AP01	Pasien membuat appointment dengan dokter.
AO-AP02	Admin mengubah status appointment.
AO-AP03	Dokter mengubah status appointment.

5.6.2 Hasil

Tabel 21: Hasil Uji Fitur Appointment

Identifikasi	Input	Hasil yang diharapkan	Hasil sebenarnya	Status
AO-AP01	Pasien memilih dokter, jadwal, dan mengisi data appointment	Sistem menyimpan data appointment pasien	Sistem berhasil menyimpan data appointment pasien	Sukses
AO-AP02	Admin mengubah status appointment	Sistem menyimpan perubahan status appointment oleh admin	Sistem berhasil menyimpan perubahan status appointment oleh admin	Sukses
AO-AP03	Dokter mengubah status appointment	Sistem menyimpan perubahan status appointment oleh dokter	Sistem berhasil menyimpan perubahan status appointment oleh dokter	Sukses

5.7 Fitur Rekam Medis

5.7.1 Kasus Uji

Tabel 22: Kasus Uji Fitur Rekam Medis

Identifikasi	Skenario
AO-RM01	Dokter membuat rekam medis pasien.
AO-RM02	Admin, dokter, dan pasien mengakses data rekam medis sesuai hak akses.

5.7.2 Hasil

Tabel 23: Hasil Uji Fitur Rekam Medis

Identifikasi	Input	Hasil yang diharapkan	Hasil sebenarnya	Status
AO-RM01	Dokter mengisi data rekam medis pasien	Sistem menyimpan data rekam medis pasien	Sistem berhasil menyimpan data rekam medis pasien	Sukses
AO-RM02	Admin, dokter, atau pasien membuka menu rekam medis	Sistem menampilkan data rekam medis sesuai hak akses pengguna	Sistem berhasil menampilkan data rekam medis sesuai hak akses pengguna	Sukses

5.8 Fitur Pembayaran

5.8.1 Kasus Uji

Tabel 24: Kasus Uji Fitur Pembayaran

Identifikasi	Skenario
AO-PB01	Admin membuat data pembayaran.
AO-PB02	Admin mengubah status pembayaran.
AO-PB03	Pasien mengakses riwayat pembayaran.

5.8.2 Hasil

Tabel 25: Hasil Uji Fitur Pembayaran

Identifikasi	Input	Hasil yang diharapkan	Hasil sebenarnya	Status
AO-PB01	Admin mengisi data pembayaran	Sistem menyimpan data pembayaran ke dalam database	Sistem berhasil menyimpan data pembayaran ke dalam database	Sukses
AO-PB02	Admin mengubah status pembayaran	Sistem menyimpan perubahan status pembayaran	Sistem berhasil menyimpan perubahan status pembayaran	Sukses
AO-PB03	Pasien membuka menu riwayat pembayaran	Sistem menampilkan riwayat pembayaran milik pasien	Sistem berhasil menampilkan riwayat pembayaran milik pasien	Sukses

6 Penutup

DAFTAR PUSTAKA

LAMPIRAN

Lampiran A Pembagian Tugas

Tabel 26: Pembagian Tugas

NIM	Nama	Tugas	Kontribusi (100%)	Penyelesaian (%)
1301223491	Alif Wide Ardiansyah	Semuanya	100%	100%

Lampiran B Source Code Penting

```
21 @Bean
22 SecurityFilterChain securityFilterChain(HttpSecurity http, JwtAuthenticationFilter jwtAuthenticationFilter)
23     throws Exception {
24     return http
25         .csrf(AbstractHttpConfigurer::disable)
26         .cors(cors -> {})
27         .httpBasic(AbstractHttpConfigurer::disable)
28         .formLogin(AbstractHttpConfigurer::disable)
29         .logout(AbstractHttpConfigurer::disable)
30         .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
31         .exceptionHandling(exception -> exception.authenticationEntryPoint(
32             new HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED)))
33         .authorizeHttpRequests(authorize -> authorize
34             .requestMatchers(
35                 "/api/v1/auth/login",
36                 "/api/v1/auth/logout",
37                 "/api/v1/health",
38                 "/api/swagger-ui/**",
39                 "/api/docs/**",
40                 "/v3/api-docs/**",
41                 "/swagger-ui/**",
42                 "/swagger-ui.html")
43             .permitAll()
44             .requestMatchers("/api/v1/auth/me")
45             .authenticated()
46             .requestMatchers("/api/v1/admin/**")
47             .hasRole("ADMIN")
48             .requestMatchers("/api/v1/doctor/**")
49             .hasRole("DOCTOR")
50             .requestMatchers("/api/v1/patient/**")
51             .hasRole("PATIENT")
52             .anyRequest()
53             .denyAll())
54         .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class)
55         .build();
56     }
57 }
```

Gambar 18: Source Code 1

```
208 private void applyRequest(Appointment appointment, AppointmentRequest request) {
209     Patient patient = patientService.findById(request.patientId());
210     Doctor doctor = doctorService.findById(request.doctorId());
211     DoctorSchedule schedule = upsertAppointmentSchedule(appointment, request);
212
213     appointment.setPatient(patient);
214     appointment.setDoctor(doctor);
215     appointment.setSchedule(schedule);
216     // The schema still has bookedAt, but the appointment time now comes from the linked schedule.
217     appointment.setBookedAt(schedule.getStartsAt());
218     appointment.setStatus(request.status() == null ? AppointmentStatus.MENUNGGU : request.status());
219     appointment.setComplaint(request.complaint());
220     appointment.setCancelledReason(request.cancelledReason());
221 }
222 }
```

Gambar 19: Source Code 2

```

58 private void applyRequest(Patient patient, PatientRequest request) {
59     patient.setFullName(request.fullName());
60     patient.setEmail(request.email());
61     patient.setPasswordHash(passwordEncoder.encode(request.password()));
62     patient.setPhoneNumber(request.phoneNumber());
63     patient.setDateOfBirth(request.dateOfBirth());
64 }

```

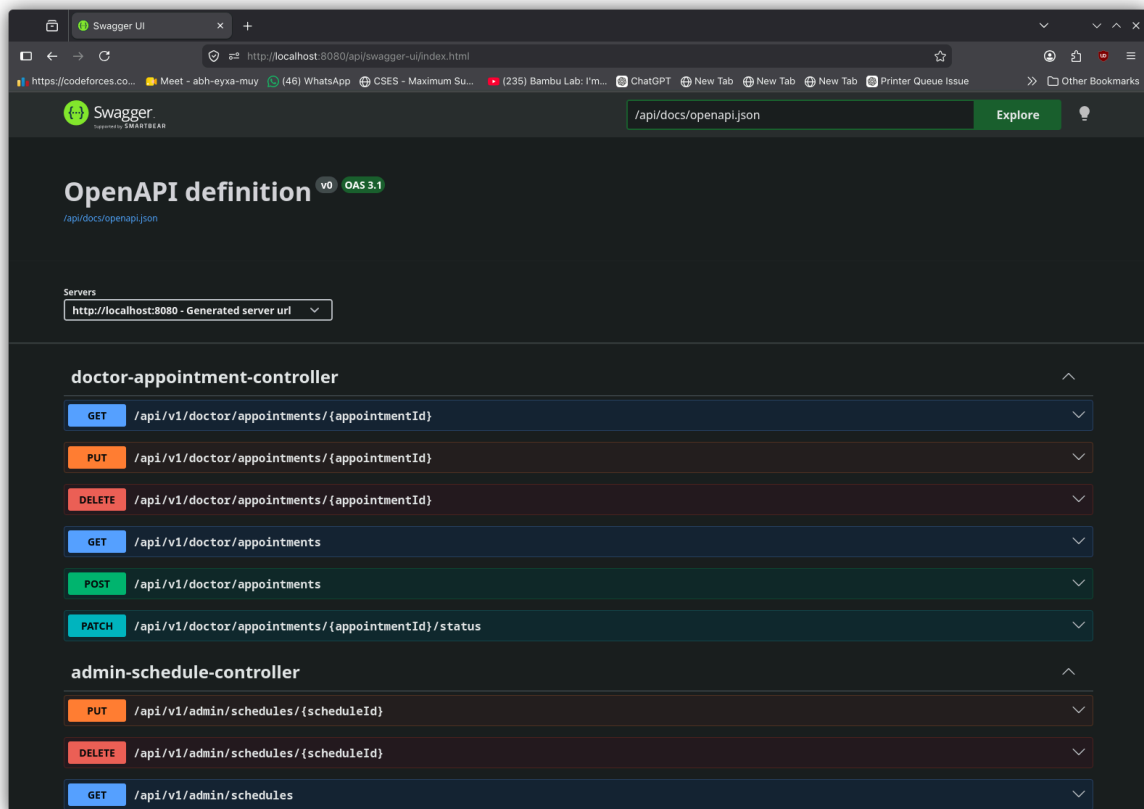
Gambar 20: Source Code 3

```

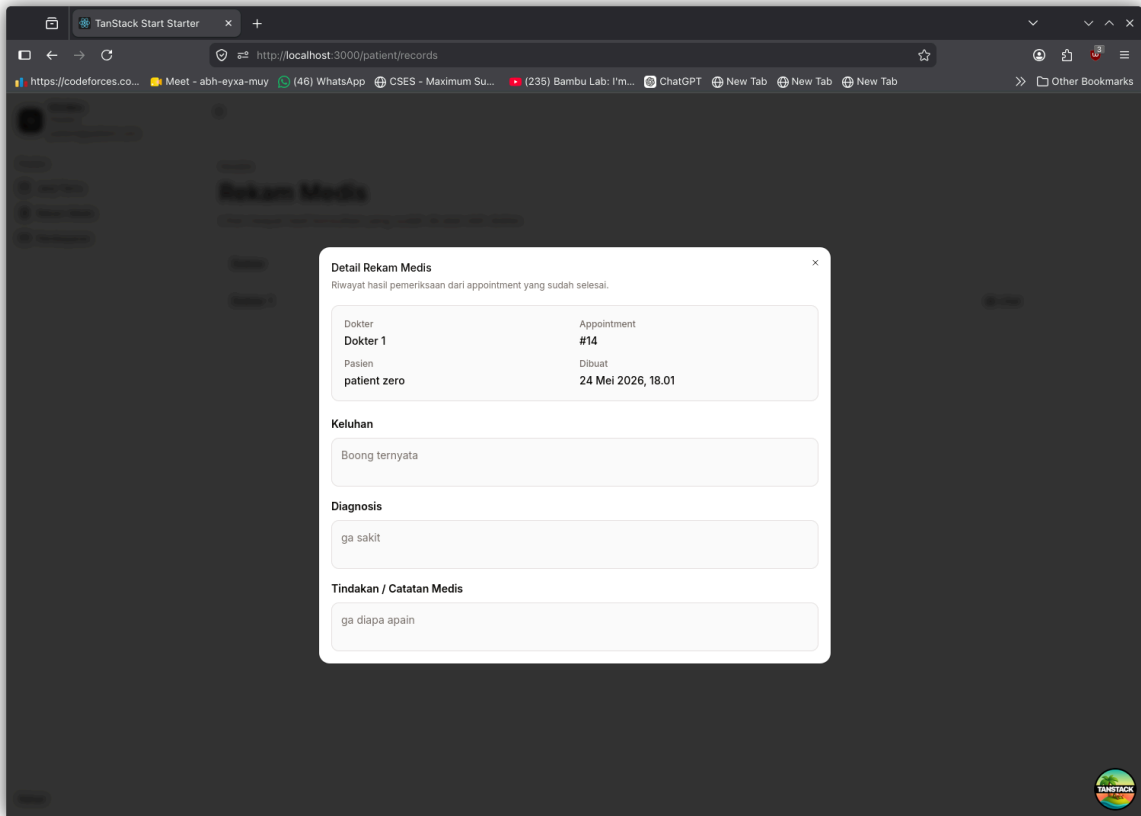
59 private void applyRequest(Doctor doctor, DoctorRequest request) {
60     ensureLicenseNumberIsAvailable(doctor, request.licenseNumber());
61     doctor.setFullName(request.fullName());
62     doctor.setEmail(request.email());
63     doctor.setPasswordHash(passwordEncoder.encode(request.password()));
64     doctor.setSpecialization(request.specialization());
65     doctor.setLicenseNumber(request.licenseNumber());
66 }
67
68 private void ensureLicenseNumberIsAvailable(Doctor doctor, String licenseNumber) {
69     doctorRepository.findByLicenseNumber(licenseNumber)
70         .filter(existingDoctor -> !Objects.equals(existingDoctor.getId(), doctor.getId()))
71         .ifPresent(existingDoctor -> {
72             throw new IllegalArgumentException("Doctor license number is already registered");
73         });
74 }

```

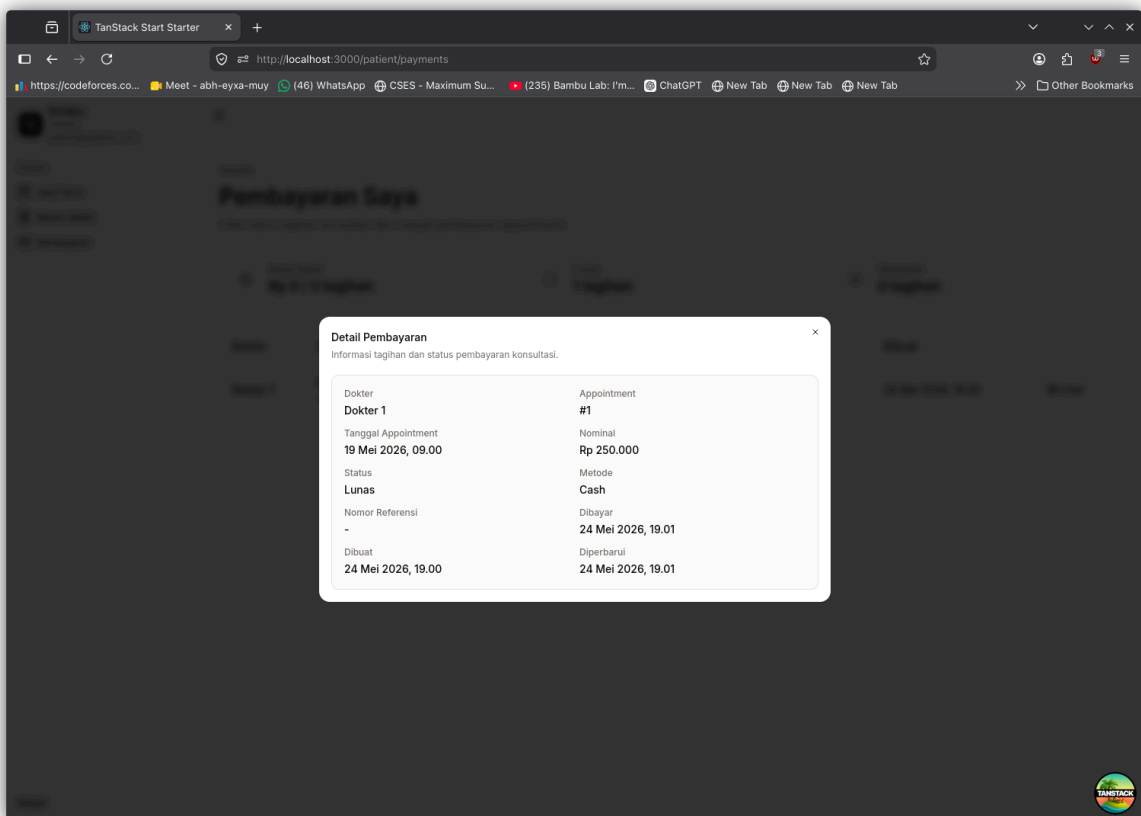
Lampiran C Screenshot Sistem



Gambar 21: Screenshot Sistem 1



Gambar 22: Screenshot Sistem 2



Gambar 23: Screenshot Sistem 3

The screenshot shows a web browser window with the URL `http://localhost:3000/doctor/appointments`. A modal titled "Tambah Appointment" is open, displaying a form for adding a new appointment. The form includes fields for Patient, Date, Room, Start Time, End Time, Status, and a Notes section. A "Tambah Appointment" button is at the bottom.

Tambah Appointment

Appointment baru akan membuat jadwal dokter berstatus booked.

Pasien: patient one

Tanggal Appointment: 06 / 17 / 2026

Ruangan:

Jam Mulai: 09 : 00 AM

Jam Selesai: 09 : 30 AM

Keluhan:

Status: Menunggu

Catatan Jadwal:

Tambah Appointment

Gambar 24: Screenshot Sistem 4

The screenshot shows the "Appointment" view in the TanStack Start Starter application. The view displays a calendar for the week of 28 Jun - 4 Jul 2026. A sidebar on the left contains navigation links for "Klinikku", "Dokter", "Jadwal Saya", and "Rekam Medis". The main area shows a calendar grid with a blue slot for "09:00 - 09:30 - patient" on Wednesday, June 30th. A "Tambah Appointment" button is visible in the top right corner.

Klinikku
Dokter
dokter1@dokter.com

Dokter

Jadwal Saya

Rekam Medis

DOKTER

Appointment

Lihat appointment Anda dan buat slot baru untuk pasien.

+ Tambah Appointment

< > today 28 Jun - 4 Jul 2026 week day

	Min 28/6	Sen 29/6	Sel 30/6	Rab 1/7	Kam 2/7	Jum 3/7	Sab 4/7
07							
08							
09			09:00 - 09:30 - patient				
10							
11							
12							
13							
14							
15							
16							
17							
18							

Keluar

Gambar 25: Screenshot Sistem 5

Buat Rekam Medis

Rekam medis akan terhubung ke appointment, pasien, dan dokter yang bertugas.

Appointment selesai

#12 - patient zero - 25 Mei 2026, 07:30

Keluhan

Sakit Punggung nya sudah sembuh yaaa

Diagnosis

sakit punggung

Tindakan / Catatan Medis

obatin

Batal Simpan Rekam Medis

Gambar 26: Screenshot Sistem 6

Klinikku Dokter
dokter1@dokter.com

Dokter

Jadwal Saya

Rekam Medis

Rekam Medis

Lihat rekam medis pasien dan buat catatan dari appointment yang sudah selesai.

+ Buat Rekam Medis

Pasien	Appointment	Diagnosis	Dibuat	
patient zero	#12	sakit punggung	16 Jun 2026, 17:15	Lihat
patient zero	#14	ga sakit	24 Mei 2026, 18:01	Lihat

Keluar

Gambar 27: Screenshot Sistem 7

Tambah Dokter

Akun dokter baru akan langsung bisa digunakan untuk masuk.

Nama Lengkap
Dokter Tawon

Email
tawon@tawon.com

Password
••••••••

Spesialisasi
Gigi

Nomor Lisensi
LV-2224151

+ Tambah Dokter

Gambar 28: Screenshot Sistem 8

ADMIN

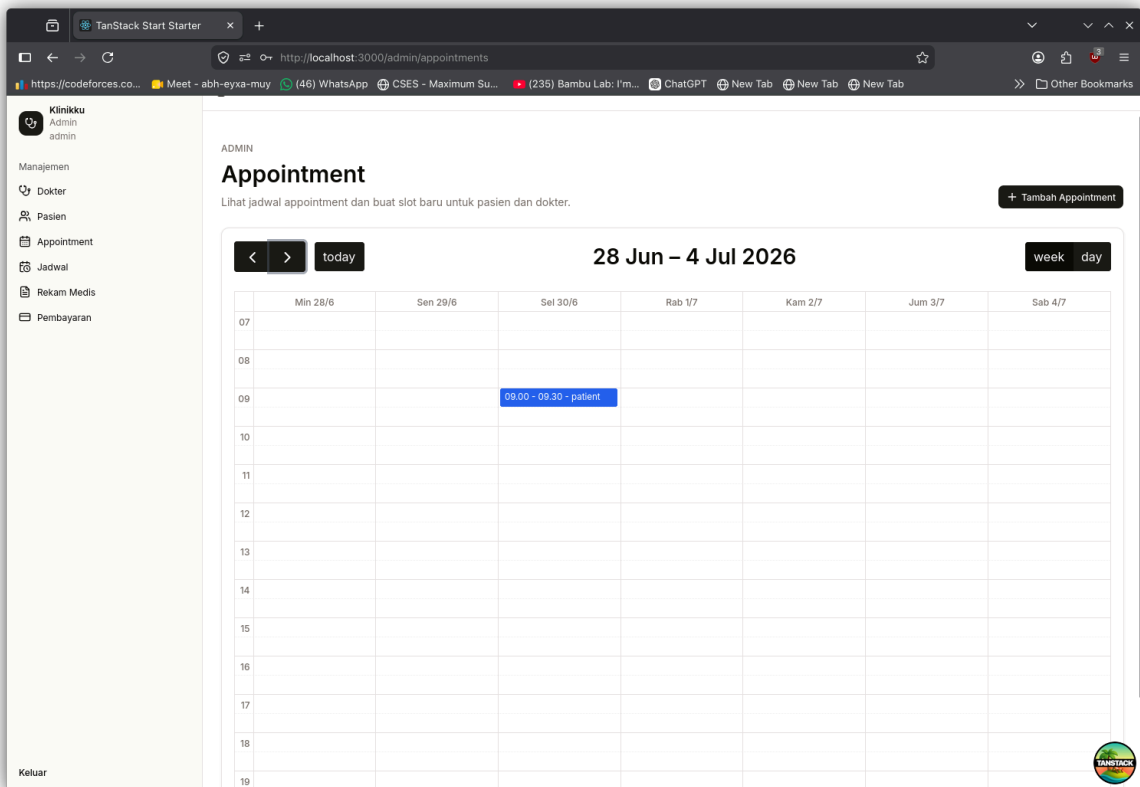
Daftar Dokter

Kelola data dokter, spesialisasi, nomor lisensi, dan akun masuk.

+ Tambah Dokter

Nama	Email	Spesialisasi	No. Lisensi	Dibuat	
Dokter 1	dokter1@dokter.com	Penyakit Dalam	2352325	18 Mei 2026	:
Dokter 2	dokter2@dokter.com	Kardiologi	2352342	18 Mei 2026	:
Dokter 3	dokter3@dokter.com	Ortho	35123532	19 Mei 2026	:
Dokter Tawon	tawon@tawon.com	Gigi	LV-2224151	16 Jun 2026	:

Gambar 29: Screenshot Sistem 9



Gambar 30: Screenshot Sistem 10

Lampiran D Manual Instalasi Sistem

1. Clone repository berikut:
<https://github.com/alipw/pbo-ta>
2. Pastikan sudah terinstall
 - Java JDK 21 atau lebih baru
 - Node.js dan npm
 - Docker
3. Versi yang sudah di test (mungkin akan ada kendala jika versi tidak sama)
 - Java 26
 - Node.js 26
 - npm 11
 - Docker 29
4. Jalankan Database

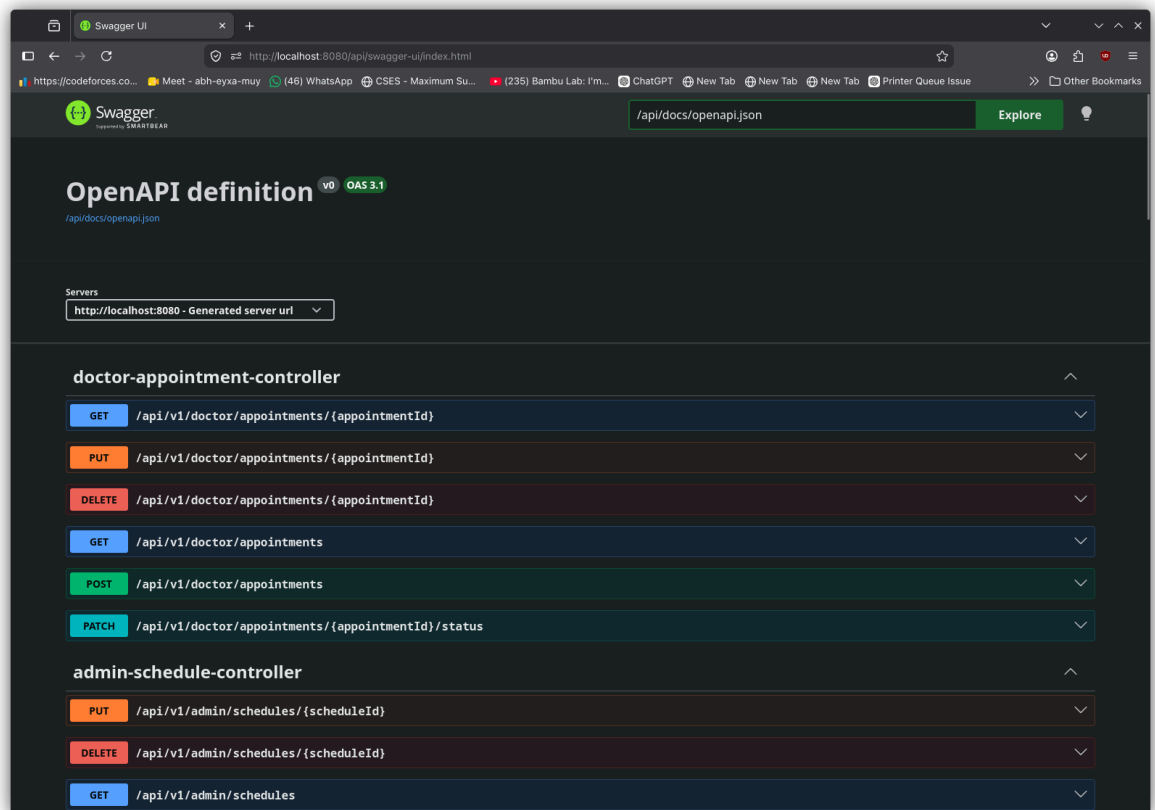
Jika semua sudah terinstall, Jalankan perintah ini

```
docker compose up -d postgres
```

Ini akan memulai (atau menginstall jika belum ada) proses postgresql yang akan digunakan oleh klinikku sebagai database dengan konfigurasi:

 - Host: `localhost`

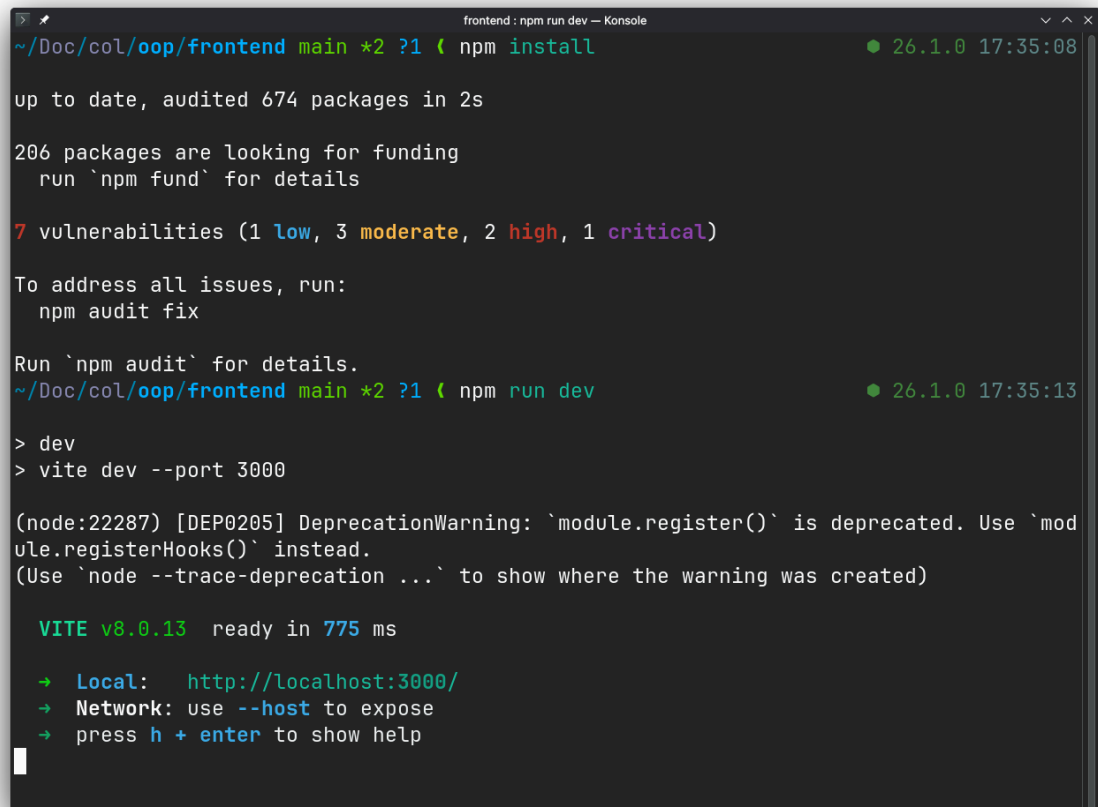
- Port: `5432`
 - Database: `klinikku`
 - Username: `postgres`
 - Password: `postgres`
5. Jalankan Backend
- masuk ke folder backend dengan perintah
- ```
cd backend
```
- siapkan env
- ```
cp .env.example .env
```
- export variable env lalu jalankan
- ```
set -a
```
- ```
source .env
```
- ```
set +a
```
- ```
./mvnw spring-boot:run
```
- Backend akan berjalan di port 8080, cek localhost:8080/api/docs utk melihat dokumentasi api



Gambar 31: Dokumentasi API

6. Jalankan Frontend
- ```
cd frontend
```
- ```
npm install
```

npm run dev



```
frontend : npm run dev — Konsole
~/Doc/col/oop/frontend main *2 ?1 ( npm install 26.1.0 17:35:08

up to date, audited 674 packages in 2s

206 packages are looking for funding
  run `npm fund` for details

7 vulnerabilities (1 low, 3 moderate, 2 high, 1 critical)

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
~/Doc/col/oop/frontend main *2 ?1 ( npm run dev 26.1.0 17:35:13

> dev
> vite dev --port 3000

(node:22287) [DEP0205] DeprecationWarning: `module.register()` is deprecated. Use `module.registerHooks()` instead.
(Use `node --trace-deprecation ...` to show where the warning was created)

VITE v8.0.13 ready in 775 ms

→ Local:   http://localhost:3000/
→ Network: use --host to expose
→ press h + enter to show help
```

Gambar 32: Tampilan Frontend

Frontend akan berjalan di port 3000

untuk dokumentasi lebih lanjut baca [README.md](#) yang sudah disediakan di project